

Master Thesis

Organ shape synthesis using octrees

Weekly Report 1

Ludvik Petersen (s194613), Martin Ægidius (s194119)

February 14, 2025

0.1 Project plan

Week	Activity	Risk
1-2	Access to necessary code, data & resources, gain codebase overview, refactoring and applying ground work for upcoming weeks	1
2-3	Introduction, literature review, OctFusion codebase-review	1
3-5	Pipeline design, VAE experimentation and start writing method section	2
6-7	VAE analysis and choice of architecture	3
8-9	VAE result reporting and further literature review	2
10-11	diffusion architecture implementation	3
12-13	Initial model training with small subsample	5
14-15	Architectural experimentation and build supporting framework for multi-organ classifier	5
16-17	Upscale to multi-organ classifier-guidance model and write imagecas results	5
18-20	Create proof-of-concept through clinical data usage and write open-source results	5
20-24	Write report and finish up codebase	2

What has been done this week

- Code base analysis
- Refactored project for own use (hydra, shell scripts and more)
- Trained DualOctree autoencoder model (to ensure everything is working as expected and we *roughly* understand what is going on)
- Begun writing an introduction, sample at bottom of this weekly report

General structure of document: sections containing questions are indicated by ending on "questions". Questions are highlighted in context with red text in other sections when a specific context applies.

Training a simple autoencoder on ImageCAS

We train a simple octree autoencoder using an octree depth of 7 and a full (bottleneck) depth of 4. As input to the autoencoder we use dual octrees built on the basis of pointcloud data paired with the corresponding normal vectors. Each encoder stage consists of a graph convolution which encodes node features (more information on this at a later stage) followed by a graph downsampling (except at the last stage at which the current graph depth is maintained). Each decoder stage utilizes an upsampling layer followed by a convolution, except for the bottleneck layer where an initial convolution is applied. Node splitting logits are calculated per decoder stage by a prediction module (explain later), which is used for growing the dual octree. In addition, at each decoder stage, the multi-level partition of unity (MPU) is utilized to regress SDF field values corresponding to the ground-truth SDF evaluated at discrete sample positions.

Losses

Outputs of the prediction module are used to formulate the octree loss (binary crossentropy):

$$\mathcal{L}_{octree} = \sum_d \frac{1}{N_d} \sum_{o_{pred} \in O_d} \text{CrossEntropy}(o_{pred}, o_{gt}) \quad (1)$$

where d is the depth of an octree layer, N_d corresponds to the number of nodes in the d 'th layer and o indicates a node status. This loss enforces learning of feature encodings which allow correct node-splitting prediction during decoding. The MPU output is compared to ground-truth SDF values using a composite L2-loss based on field-values and gradients:

$$\mathcal{L}_{regress} = \sum_d \frac{1}{N_{\mathcal{P}}} \sum_{x \in \mathcal{P}} \underbrace{(\lambda_v \|F(x) - G(x)\|_2^2)}_{\mathcal{L}_{SDF_d}} + \underbrace{\|\nabla F(x) - \nabla G(x)\|_2^2}_{\mathcal{L}_{grad_d}} \quad (2)$$

where x is a sample position, $\mathcal{P}$ denotes the collection of points at which the ground-truth SDF has been sampled, $F(x)$ and $G(x)$ are the MPU-inferred and

ground-truth SDF values, $\nabla F(x)$ is calculated using automatic differentiation and $\nabla_{x,y,z} G(x)$ is the normalized outward-pointing vector **from the closest surface to the sampled point**.

The balancing constant λ_v was set to 200. $\mathcal{L}_{SDF}$ is used to enforce correct absolute placement of the volumetric field with respect to the sampled position, which allows correct shape representation. Using this loss alone would possibly allow producing correct placement of the volume, but the minimizing weights would produce fields which do not necessarily behave as expected in points $x \notin \mathcal{P}$ as the learned field is would not be guaranteed consistently continuous. Thus, to enforce the implicit field representation to be globally coherent (and therefore generalizable), $\mathcal{L}_{grad}$ is utilized to learn ground-truth corresponding changes in SDF-values represented by the normalized outward-pointing vector **from the closest surface to the sampled point**, which is expected to allow proper interpolation for $x \notin \mathcal{P}$ from learned field values $SDF(x), x \in \mathcal{P}$.

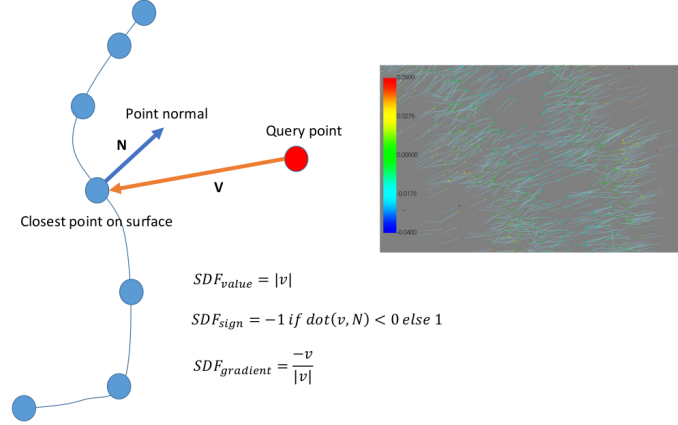


Figure 1: Why do we use this method for gradient calculation rather than using finite differences (as author does)? Is it only due to computational efficiency? Are we guaranteed that the gradient actually points in this direction along the whole path?

Autoencoder training results

A selection of training and testing results are presented in Figures 2 and 6.

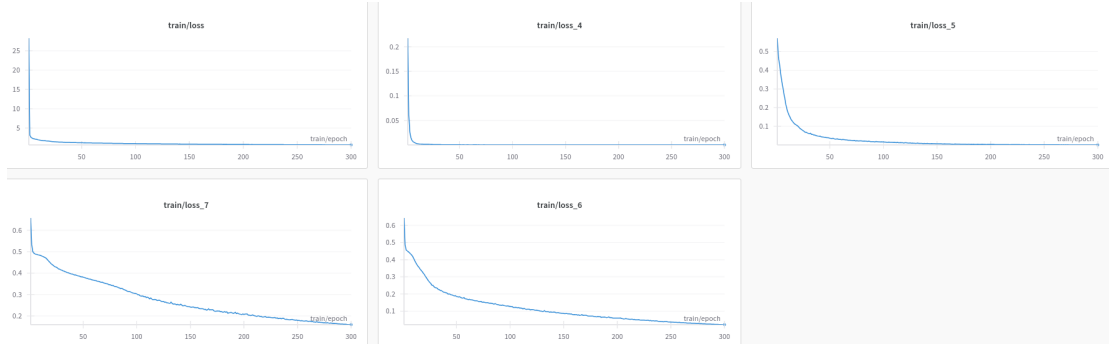


Figure 2: A selection of train-losses

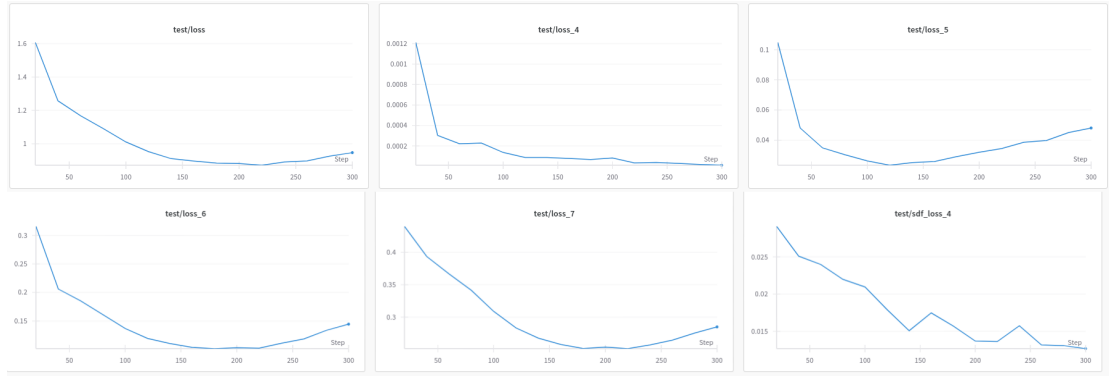


Figure 3: A selection of test-losses

We notice that losses are larger when the layer-number is higher. Our understanding is that this is because higher layer-numbers correspond to a higher resolution surface, which thus is a harder task (and obviously depends on correctness of the grown octrees, which we expect has a larger number of errors the higher stage of decoding we use). *Is this also your understanding?*

Codebase questions

```
class TransformShape:
    def __init__(self, flags):
        self.flags = flags

        self.point_sample_num = 3000
        self.sdf_sample_num = 5000
        self.points_scale = 0.5 # the points are in [-0.5, 0.5]
        self.noise_std = 0.005

        self.depth = flags.depth
        self.full_depth = flags.full_depth

    def points2octree(self, points: Points):
        octree = Octree(self.depth, self.full_depth)
        octree.build_octree(points)
        return octree

    def process_points_cloud(self, sample):
        # get the input
        points, normals = sample['points'], sample['normals']
        points = points / self.points_scale # scale to [-1.0, 1.0]

        # transform points to octree
        points_gt = Points(points=torch.from_numpy(points).float(),
                           normals=torch.from_numpy(normals).float())
        points_gt.clip(min=-1, max=1)
        octree_gt = self.points2octree(points_gt)

        if self.flags.distort:
            # randomly sample points and add noise
            # Since we rescale points to [-1.0, 1.0] in Line 24, we also need to
            # rescale the 'noise_std' here to make sure the 'noise_std' is always
            # 0.5% of the bounding box size.
            noise_std = self.noise_std / self.points_scale
            noise = noise_std * np.random.randn(self.point_sample_num, 3)
            rand_idx = np.random.choice(points.shape[0], size=self.point_sample_num) # Q: Why shuffle?
            points_noise = points[rand_idx] + noise

            points_in = Points(points=torch.from_numpy(points_noise).float(),
                               features=torch.ones(self.point_sample_num, 1).float())
            points_in.clip(-1.0, 1.0)
            octree_in = self.points2octree(points_in)
        else:
            points_in = points_gt
            octree_in = octree_gt

        # construct the output dict
        return {'octree_in': octree_in, 'points_in': points_in.points,
               'octree_gt': octree_gt, 'points_gt': points_gt}
```

Figure 4: Why do we shuffle idx's? We believe it is because we only distort parts of the image, as `point_sample_num` is probably $< \text{len}(\text{points})$

- See Figure 6 caption
- Use of GraphVAE's in their code. Perform experiments, but only reference AE in paper. Use this as basis for our generative model architecture? Do OctFusion also use this framework?
- Losses are higher when the "number" is higher in the loss-name - is it also your understanding that this corresponds to higher decoder stages, and thus higher surface resolution representation?
- In dual octree paper he uses finite difference to calculate $\nabla G(x)$. Rasmus uses $\nabla G(x) = \frac{-v}{|v|}$ (see Figure 1). Why do we use this method for gradient calculation rather than using finite differences? Is it only due to computational efficiency? Are we actually guaranteed that the gradient actually points in this direction along the whole path?

0.1.1 Autoencoder reconstruction results

Having trained our model, we're able to reproduce results that are of similar quality to Rasmus's results. Output would not get perfect IOU performance, as is evident by the wireframe outlines, which both fill void space as well as fail to encompass the whole geometric shape. This is expected and something we can evaluate our generative output on.

Question

- Are there some metrics that you deem especially valuable for our task? Volumetric IOU, Chamfer Distance, Normal Consistency and F-Score are used in the dual-octree paper. Dice Score/SSIM/Hausdorff Distance?

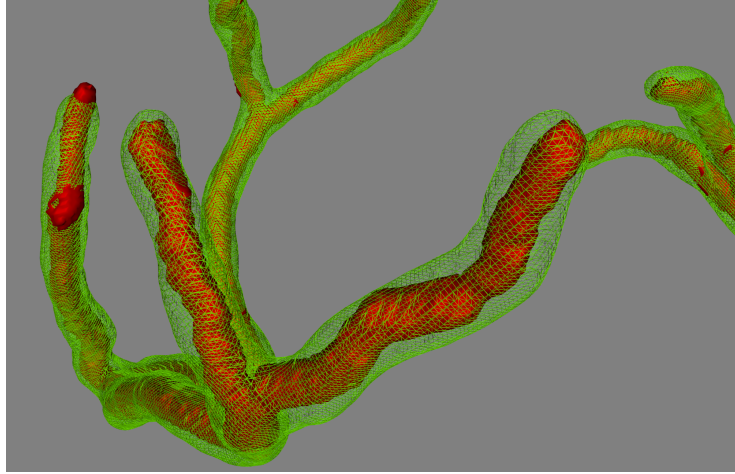


Figure 5: Reconstruction result (wireframe outline) vs ground truth of sample 891

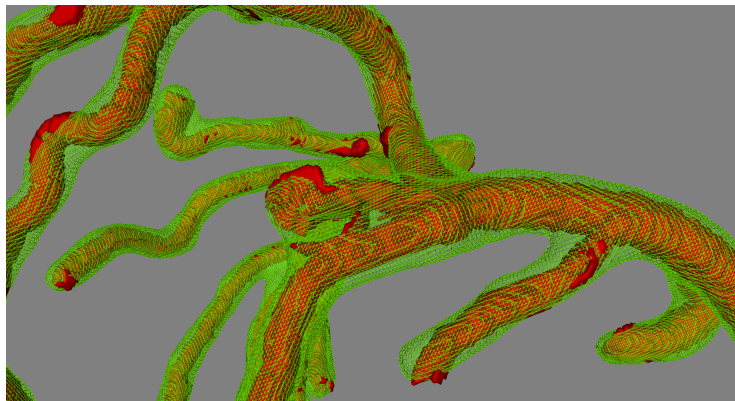


Figure 6: RAPA reconstruction result performance (wireframe) on sample 7

General questions

- We are having a hard time writing the introduction (following section), as we are a bit unsure that we understand the overall story we're trying to tell.

Currently the introduction is based a lot on synthetic CT scans, which obviously is not what we are doing. We are having a hard time finding relevant work in the field of organ shape synthesis, and thus have a hard time formulating the clinical background.

A possible idea could be to be able to synthesize a set of organ shapes (corresponding to segmentations) and use these in a conditional CT image generation pipeline to regularize the process (but this is maybe a bit farfetched...)

Does shape synthesis of arteries in itself hold some sort of clinical value, or would value only be in downstream tasks of some sort?.

Project introduction current progress

The advent of deep learning based methods have recently gained traction in the field of medical image analysis, partly due to increased representational ability and flexibility to various downstream tasks as compared to more traditional machine learning methods. One potential use case is disease detection for ischemic heart disease (IHD) based on Computed tomography angiography (CTA) data. 2021 mortality estimates identified 9.21 million global deaths due to IHD [2]. IHD can e.g. be caused by coronary artery stenosis, which may be identified using CTA paired with deep learning based image segmentation, which allows quick diagnosis by radiologists and clinicians.

Such an approach generally comes at the cost of data burden and significant uses of compute for model training. In the general case, deep learning models exhibit a lower data-efficiency than traditional machine learning approaches. This poses a large problem in the field of medical imaging where large-scale image acquisition is notoriously difficult, partly due to economic factors and adverse health effects for patients due to exposure to ionizing radiation during the imaging procedure and patient privacy concerns. Findings from privacy research has complicated the landscape further, as information leakage may be present in deep learning methods **find inversion attack medical imaging relevant citation**.

A potential solution to the data burden is to generate synthetic Computed Tomography (sCT) datasets which may be utilized in downstream tasks. The field of deep learning based sCT has historically been dominated by CNNs and GANs[1]. Exciting development in stable diffusion models has begun to be utilized in sCT **FIND GOOD CITATION**, but are generally intractable due to the computational complexity of the diffusion process in voxel-space. Utilizing latent stable diffusion is a natural solution to minimize the computational complexity of a diffusion process, but generating a well-structured and descriptive latent space while minimizing the computational budget is non-trivial.

Authors of [3] propose to use dual octrees as an efficient geometric representation for training deep networks for volumetric shape representation. This thesis explores the possibility of realistic generative coronary artery volume representations using latent diffusion based on dual octrees.

Project status according to the study plan

Currently on schedule

Plan for the next weeks

1. Finish first version of introduction section
2. Literature reading
3. Define evaluation metrics and implement them
4. OctFusion code base review - see how hard to understand it is and if there at least are parts for inspiration
5. Starting formulating a VAE-framework

References

- [1] Sanuwani Dayarathna, Kh Tohidul Islam, Sergio Uribe, Guang Yang, Munawar Hayat, and Zhaolin Chen. Deep learning based synthesis of mri, ct and pet: Review and analysis. *Medical Image Analysis*, 92:103046, 2024.
- [2] Seth S Martin, Aaron W Aday, Zaid I Almarzooq, Cheryl AM Anderson, Pankaj Arora, Christy L Avery, Carissa M Baker-Smith, Bethany Barone Gibbs, Andrea Z Beaton, Amelia K Boehme, et al. 2024 heart disease and stroke statistics: a report of us and global data from the american heart association. *Circulation*, 149(8):e347–e913, 2024.
- [3] Peng-Shuai Wang, Yang Liu, and Xin Tong. Dual octree graph networks for learning adaptive volumetric shape representations. *ACM Transactions on Graphics*, 41(4):1–15, July 2022.